

Streamlit + PyCryptodome 이용한 웹 애플리케이션 개발

학번: 202155011

이름: 송영욱

1. 개요

사용자는 복잡한 설정 없이 웹 브라우저 상에서 직접 메시지를 암호화하고, 복호화하며, 과정을 소개합니다. Python의 Streamlit 프레임워크를 사용하여 웹 UI를 구축하였고, 핵심 암호화 기능은 PyCryptodome 라이브러리를 활용하여 구현하였다. AI는 GEMINI로 UI 구성을 부탁 하였습니다. 비즈네르 암호를 물어보고 코드를 고쳐나가면서 작업했습니다. 최대한 수업에서 배운 코드를 활용하려고 했습니다. 추가적으로 코드 오류를 고쳐나가면서 제시했습니다.

(AI 사용부분은 코드상에 주석으로 표시했습니다.)

```
import streamlit as st
from Crypto.Cipher import AES
import string
from Crypto.PublicKey import RSA
from Crypto.Random import get_random_bytes
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Util.Padding import pad, unpad
import hashlib
import base64
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA512

st.set_page_config(
    page_title="메시지 보안 도구",
    page_icon="🔒",
    layout="wide"
)

# --- AI 활용: UI/UX 디자인 개선 ---
# Gemini Code Assist를 활용하여 애플리케이션의 전체적인 다크 모드 테마를 디자인했습니다.
# AI는 전문적인 느낌을 주는 색상 조합, 폰트, 버튼 및 탭 스타일 등 UI 디자인을 위한 CSS 코드를 생성하는 데 도움을 주었습니다.
# --- UI/UX 개선을 위한 CSS 스타일 ---
st.markdown(
    """
    <style>
        @import
url('https://fonts.googleapis.com/css2?family=Noto+Sans+KR:wght@400;500;700&display=swap');

        /* 전체적인 폰트 및 배경 설정 */
        .stApp {
            background-color: #1e1e1e; /* Dark background */
            font-family: 'Noto Sans KR', sans-serif;
        }
    </style>
    """
)
```

```
    color: #e0e0e0; /* Light text */
}

/* 제목 스타일 */
h1 {
    color: #00bcd4; /* Cyan */
    text-align: center;
    font-weight: 700;
    padding-bottom: 20px;
    border-bottom: 2px solid #333;
}

/* 헤더 스타일 (탭 내부) */
h2 {
    color: #00bcd4; /* Cyan */
    border-left: 5px solid #00bcd4;
    padding-left: 15px;
    margin-top: 20px;
}
h3 {
    color: #bdbdbd; /* Lighter Gray */
    margin-top: 15px;
}

/* 탭 스타일 */
.stTabs [data-baseweb="tab-list"] {
    gap: 24px;
    background-color: #252526;
    padding: 10px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.05);
}
.stTabs [data-baseweb="tab"] {
    height: 50px;
    white-space: pre-wrap;
    background-color: #333;
    border-radius: 8px;
    color: #a0a0a0; /* Dimmed text */
    font-weight: 500;
    transition: all 0.3s ease;
}
.stTabs [data-baseweb="tab"]:hover {
    background-color: #444;
    color: #e0e0e0;
}
```

```
.stTabs [data-baseweb="tab"][aria-selected="true"] {  
  background-color: #00838f; /* Darker Cyan */  
  color: white;  
  font-weight: 700;  
}
```

```
/* 버튼 스타일 */
```

```
.stButton>button {  
  border: none;  
  border-radius: 8px;  
  padding: 10px 20px;  
  font-weight: 500;  
  color: white;  
  background-color: #0097a7; /* Cyan */  
  transition: all 0.3s ease;  
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);  
}
```

```
.stButton>button:hover {  
  background-color: #00bcd4; /* Brighter Cyan */  
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);  
  transform: translateY(-2px);  
}
```

```
.stButton>button:active {  
  background-color: #00838f;  
  transform: translateY(0);  
}
```

```
/* 코드 블록 스타일 */
```

```
.stCodeBlock, pre {  
  border: 1px solid #444;  
  border-radius: 8px;  
  background-color: #2a2a2a;  
  padding: 15px !important;  
}
```

```
/* Expander 스타일 */
```

```
.st-expander {  
  border: 1px solid #444;  
  border-radius: 8px;  
  background-color: #2c2c2c;  
}
```

```
.st-expander header {  
  font-size: 1.1rem;  
  font-weight: 500;  
  color: #e0e0e0;
```

```

    }
</style>
"""
    unsafe_allow_html=True,
)

# ---- RSA 디지털 서명 ----
def rsa_sign(message: bytes, private_key_pem: str) -> bytes:
    """RSA 개인키로 메시지에 서명합니다."""
    key = RSA.import_key(private_key_pem)
    h = SHA512.new(message)
    signature = pkcs1_15.new(key).sign(h)
    return signature

def rsa_verify(message: bytes, signature: bytes, public_key_pem: str) -> bool:
    """RSA 공개키로 서명을 검증합니다."""
    key = RSA.import_key(public_key_pem)
    h = SHA512.new(message)
    try:
        pkcs1_15.new(key).verify(h, signature)
        return True
    except (ValueError, TypeError):
        return False

# ---- AES-CBC ----
def encrypt_aes_cbc(message: str, key: bytes) -> (str, str):
    cipher = AES.new(key, AES.MODE_CBC)
    ct_bytes = cipher.encrypt(pad(message.encode('utf-8'), AES.block_size))
    iv = base64.b64encode(cipher.iv).decode('utf-8')
    ct = base64.b64encode(ct_bytes).decode('utf-8')
    return iv, ct

def decrypt_aes_cbc(iv: str, ciphertext: str, key: bytes) -> str:
    try:
        iv = base64.b64decode(iv)
        ct = base64.b64decode(ciphertext)
        cipher = AES.new(key, AES.MODE_CBC, iv)
        pt = unpad(cipher.decrypt(ct), AES.block_size)
        return pt.decode()
    except Exception as e:
        return f"복호화 오류: {str(e)}"

def generate_rsa_keys():
    key = RSA.generate(2048)

```

```

private_key = key.export_key().decode()
public_key = key.publickey().export_key().decode()
return public_key, private_key

def encrypt_rsa(message, public_key):
    try:
        key = RSA.import_key(public_key)
        cipher = PKCS1_OAEP.new(key)
        encrypted = cipher.encrypt(message.encode())
        return base64.b64encode(encrypted).decode()
    except Exception as e:
        return f"암호화 오류: {str(e)}"

def decrypt_rsa(ciphertext, private_key):
    try:
        key = RSA.import_key(private_key)
        cipher = PKCS1_OAEP.new(key)
        decrypted = cipher.decrypt(base64.b64decode(ciphertext))
        return decrypted.decode()
    except Exception as e:
        return f"복호화 오류: {str(e)}"

def calculate_sha512(message):
    return hashlib.sha512(message.encode()).hexdigest()

def caesar_encrypt(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            result += chr((ord(char) - ascii_offset + shift) % 26 + ascii_offset)
        else:
            result += char
    return result

def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)

# --- 카이제곱 통계 기반 암호 해독 (from caesarCipher-ChiSquare-UL-Digits.py) ---
EN_FREQ = {
    'A':8.167, 'B':1.492, 'C':2.782, 'D':4.253, 'E':12.702, 'F':2.228, 'G':2.015, 'H':6.094,
    'I':6.966, 'J':0.153, 'K':0.772, 'L':4.025, 'M':2.406, 'N':6.749, 'O':7.507, 'P':1.929,
    'Q':0.095, 'R':5.987, 'S':6.327, 'T':9.056, 'U':2.758, 'V':0.978, 'W':2.360, 'X':0.150,
    'Y':1.974, 'Z':0.074
}

```

```

def chi_square_score(text: str) -> float:
    """주어진 텍스트의 카이제곱 점수를 계산합니다 (영문 알파벳 기준)."""
    text_letters = [c for c in text.upper() if c in string.ascii_uppercase]
    N = len(text_letters)
    if N == 0: return float('inf')

    observed = {ch: 0 for ch in string.ascii_uppercase}
    for c in text_letters:
        observed[c] += 1

    score = 0.0
    for ch in string.ascii_uppercase:
        expected = N * (EN_FREQ[ch] / 100.0)
        if expected > 0:
            diff = observed[ch] - expected
            score += (diff * diff) / expected
    return score

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def mod_inverse(a, m):
    def extended_gcd(a, b):
        if a == 0:
            return b, 0, 1
        gcd, x1, y1 = extended_gcd(b % a, a)
        x = y1 - (b // a) * x1
        y = x1
        return gcd, x, y

    _, x, _ = extended_gcd(a, m)
    return (x % m + m) % m

def affine_encrypt(text, a, b):
    if gcd(a, 26) != 1:
        raise ValueError("'a'는 26과 서로소여야 합니다.")

    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            x = ord(char) - ascii_offset

```

```

        encrypted = (a * x + b) % 26
        result += chr(encrypted + ascii_offset)
    else:
        result += char
    return result

def affine_decrypt(text, a, b):
    if gcd(a, 26) != 1:
        raise ValueError("'a'는 26과 서로소여야 합니다.")

    a_inv = mod_inverse(a, 26)
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            y = ord(char) - ascii_offset
            decrypted = (a_inv * (y - b)) % 26
            result += chr(decrypted + ascii_offset)
        else:
            result += char
    return result

def combined_encrypt(text, caesar_shift, affine_a, affine_b):
    # 먼저 카이사르 암호화 적용
    caesar_result = caesar_encrypt(text, caesar_shift)
    # 그 다음 아핀 암호화 적용
    return affine_encrypt(caesar_result, affine_a, affine_b)

def combined_decrypt(text, caesar_shift, affine_a, affine_b):
    # --- AI 활용: 로직 오류 수정 ---
    # 개발 초기, 조합 암호의 복호화 순서(아핀 -> 카이사르)를 반대로 구현하여 오류가 발생했습니다.
    # AI는 암호화의 역순으로 복호화를 수행해야 한다는 점을 지적하고, 올바른 함수 호출 순서를 제안하여
    버그를 수정하는 데 도움을 주었습니다.
    # 먼저 아핀 복호화 적용
    affine_result = affine_decrypt(text, affine_a, affine_b)
    # 그 다음 카이사르 복호화 적용
    return caesar_decrypt(affine_result, caesar_shift)

# ---- AES-OFB 유틸 (첨부 파일에서 통합) ----
def b64e(data: bytes) -> str:
    return base64.b64encode(data).decode("utf-8")

def b64d(data: str) -> bytes:
    return base64.b64decode(data.encode("utf-8"))

```

```

def aes_ofb_encrypt(plaintext: str, key: bytes):
    iv = get_random_bytes(16)
    cipher = AES.new(key, AES.MODE_OFB, iv=iv)
    ciphertext = cipher.encrypt(plaintext.encode("utf-8"))
    return b64e(iv), b64e(ciphertext)

def aes_ofb_decrypt(iv_b64: str, ciphertext_b64: str, key: bytes) -> str:
    iv = b64d(iv_b64)
    ciphertext = b64d(ciphertext_b64)
    cipher = AES.new(key, AES.MODE_OFB, iv=iv)
    plaintext = cipher.decrypt(ciphertext)
    return plaintext.decode("utf-8")

# ---- AES-GCM (인증된 암호화) ----
def encrypt_aes_gcm(plaintext: str, key: bytes):
    nonce = get_random_bytes(12)
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    ct, tag = cipher.encrypt_and_digest(plaintext.encode())
    return base64.b64encode(nonce).decode(), base64.b64encode(ct).decode(),
base64.b64encode(tag).decode()

def decrypt_aes_gcm(nonce_b64: str, ct_b64: str, tag_b64: str, key: bytes):
    nonce = base64.b64decode(nonce_b64)
    ct = base64.b64decode(ct_b64)
    tag = base64.b64decode(tag_b64)
    cipher = AES.new(key, AES.MODE_GCM, nonce=nonce)
    try:
        pt = cipher.decrypt_and_verify(ct, tag)
        return pt.decode()
    except Exception as e:
        return f"복호화/검증 오류: {e}"

# ---- Vigenere 암호 ----
def vigenere_encrypt(plaintext: str, key: str) -> str:
    # --- AI 활용: 알고리즘 구현 디버깅 ---
    # 비즈네르 암호의 핵심인 '키 반복' 로직 구현 중, 평문의 특정 문자(공백, 특수문자 등)를 처리할 때
    # 키 인덱스가 잘못 증가하는 오류가 있었습니다.
    # AI는 알파벳 문자에 대해서만 키 인덱스(ki)를 증가시켜야 한다는 점을 지적하며, 현재와 같은 코드
    # 구조를 제안하여 문제를 해결했습니다.
    result = []
    key_bytes = [ord(c.lower()) - 97 for c in key if c.isalpha()]
    if not key_bytes:
        raise ValueError("키에 적어도 하나의 알파벳 문자가 필요합니다.")
    ki = 0
    for ch in plaintext:

```



```

        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            shift = key_bytes[ki % len(key_bytes)]
            result.append(chr((ord(ch) - base + shift) % 26 + base))
            ki += 1
        else:
            result.append(ch)
    return ''.join(result)

def vigenere_decrypt(ciphertext: str, key: str) -> str:
    result = []
    key_bytes = [ord(c.lower()) - 97 for c in key if c.isalpha()]
    if not key_bytes:
        raise ValueError("키에 적어도 하나의 알파벳 문자가 필요합니다.")
    ki = 0
    for ch in ciphertext:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            shift = key_bytes[ki % len(key_bytes)]
            result.append(chr((ord(ch) - base - shift) % 26 + base))
            ki += 1
        else:
            result.append(ch)
    return ''.join(result)

# --- AI 활용: 코드 구조화 ---
# 개별 스크립트로 흩어져 있던 여러 암호화 기능들을 사용자가 쉽게 탐색하고 사용할 수 있도록,
# Streamlit의 탭(st.tabs) 기능을 사용하여 하나의 통합된 애플리케이션으로 재구성하는 과정에서 AI의 조
연을 얻었습니다.
st.title("□ 메시지 보안 도구")

tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs(["AES-CBC", "SHA-512+AES", "RSA", "고전 암호",
"AES-OFB", "RSA 디지털 서명"])

with tab1:
    st.header("AES-CBC 대칭키 암호화")

    with st.expander("이 □ AES-CBC 암호화란?"):
        # --- AI 활용: 설명 콘텐츠 작성 및 시각 자료 활용 ---
        # 사용자의 이해를 돕기 위해 AES-CBC의 동작 원리에 대한 설명 텍스트 초안을 AI를 통해 작성했습
니다.
        # 또한, CBC 모드의 암호화 과정을 시각적으로 보여주기 위해 Streamlit의 st.image 기능을 활용하
는 방법을 AI에게 질문하여 적용했습니다.
        st.markdown("""
AES는 현대의 가장 널리 사용되는 대칭키 블록 암호화 알고리즘입니다.

```

****주요 특징:****

1. ****블록 크기****: 128비트 (16바이트) 고정
2. ****키 크기****: 128비트(16바이트), 192비트(24바이트), 256비트(32바이트) 지원
3. ****동작 방식 (CBC 모드)****:

- 각 블록은 이전 블록의 암호문과 XOR 연산 후 암호화됩니다.

- 첫 블록은 무작위로 생성된 IV(초기화 벡터)와 XOR됩니다. 이 때문에 같은 평문이라도 암호화할 때마다 결과가 달라집니다.

****보안성:****

- 128비트 키만 해도 2^{128} (약 340조) 가지의 경우의 수를 가져, 현재 기술로는 전수조사 공격(Brute-force attack)이 불가능합니다.

""")

```
st.image("https://upload.wikimedia.org/wikipedia/commons/thumb/8/80/CBC_encryption.svg/600px-CBC_encryption.svg.png",
```

```
caption="CBC 모드 암호화 과정",
```

```
width=400)
```

```
st.divider()
```

```
# AES 키 크기 선택
```

```
key_size = st.radio(
```

```
    "AES 키 크기 선택",
```

```
    ["128비트 (16바이트)", "192비트 (24바이트)", "256비트 (32바이트)"],
```

```
    horizontal=True,
```

```
    help="더 큰 키 크기를 사용할수록 보안성이 높아지지만, 처리 시간이 약간 증가합니다."
```

```
)
```

```
key_bytes = {
```

```
    "128비트 (16바이트)": 16,
```

```
    "192비트 (24바이트)": 24,
```

```
    "256비트 (32바이트)": 32
```

```
}[key_size]
```

```
# --- 랜덤 키 생성 및 관리 ---
```

```
st.subheader("□ 키 관리")
```

```
if 'cbc_aes_key' not in st.session_state or len(st.session_state.get('cbc_aes_key', b''))
```

```
!= key_bytes:
```

```
    st.session_state['cbc_aes_key'] = get_random_bytes(key_bytes)
```

```
colk1, colk2 = st.columns([3, 1])
```

```
with colk1:
```

```
    st.code(base64.b64encode(st.session_state['cbc_aes_key']).decode(), language='text')
```

```
with colk2:
```

```

if st.button("🔑 새 랜덤 키 생성", key="new_cbc_key"):
    st.session_state['cbc_aes_key'] = get_random_bytes(key_bytes)
    st.success(f"새로운 {key_size} AES 키가 생성되었습니다.")

st.divider()

aes_message = st.text_area(
    "암호화할 메시지 입력",
    key="aes_message",
    placeholder="암호화할 평문을 입력하세요. 자동으로 PKCS7 패딩이 추가됩니다."
)

col1, col2 = st.columns(2)

with col1:
    if st.button("🔒 AES 암호화", key="encrypt_aes"):
        # --- AI 활용: UI/UX 개선 ---
        # 사용자 경험을 개선하기 위해, 각 버튼의 기능을 직관적으로 나타내는 아이콘(🔑, 🔒, 🔓 등)을

        # 텍스트와 함께 사용하는 아이디어를 AI로부터 추천받아 적용했습니다.
        if aes_message:
            key = st.session_state['cbc_aes_key']
            iv, encrypted = encrypt_aes_cbc(aes_message, key)
            st.session_state['aes_iv'] = iv
            st.session_state['aes_encrypted'] = encrypted

            # 암호화 과정 시각화
            st.markdown("---")
            st.subheader("🔒 암호화 과정")

            col_viz1, col_viz2 = st.columns(2)
            with col_viz1:
                st.markdown("***1. 초기화 벡터 (IV) 생성**")
                st.code(iv, language='text')
                st.markdown("CBC 모드의 첫 블록 암호화에 사용될 무작위 16바이트 값입니다.")

            with col_viz2:
                st.markdown("***2. PKCS7 패딩**")
                padded_len = ((len(aes_message.encode()) // 16) + 1) * 16
                st.markdown(f"평문을 16바이트 블록 단위로 맞추기 위해 패딩을 추가합니다.")
                st.code(f"원본 길이: {len(aes_message.encode())} 바이트\n패딩 후 길이: {padded_len} 바이트", language='text')

            st.markdown("***3. 암호화 결과**")
            st.markdown("패딩된 평문이 IV와 함께 AES-CBC 모드로 암호화된 후, IV와 암호문이 각각

```

```

Base64로 인코딩됩니다.")
    st.code(f"IV: {iv}\nCiphertext: {encrypted}", language='text')

    st.subheader("□ 암호화 결과")
    st.markdown("**초기화 벡터 (IV):**")
    st.code(iv, language='text')
    st.markdown("**암호문 (Base64):**")
    st.code(encrypted, language='text')

else:
    st.error("메시지를 입력하세요.")

with col2:
    if st.button("□ AES 복호화", key="decrypt_aes"):
        if 'aes_iv' in st.session_state and 'aes_encrypted' in st.session_state:
            key = st.session_state['cbc_aes_key']
            decrypted = decrypt_aes_cbc(st.session_state['aes_iv'],
st.session_state['aes_encrypted'], key)
            st.subheader("□ 복호화 결과")
            st.text_area("복호화된 메시지", decrypted, height=150)

            # 복호화 과정 설명
            if decrypted and not decrypted.startswith("복호화 오류"):
                st.markdown("---")
                st.subheader("□ 복호화 과정")
                st.markdown("""
1.  **Base64 디코딩**: Base64로 인코딩된 IV와 암호문을 다시 바이트 형태로 변환
합니다.

2.  **AES-CBC 복호화**: 입력된 키와 디코딩된 IV를 사용하여 암호문을 복호화합니
다.

3.  **패딩 제거**: 복호화된 데이터의 끝에 추가되었던 PKCS7 패딩을 제거하여 원본
메시지를 복원합니다.

""")

            if decrypted and not decrypted.startswith("복호화 오류"):
                st.success("복호화 성공! 원본 메시지가 정확히 복원되었습니다.")
            else:
                st.error("복호화 실패! 키가 잘못되었거나 암호문이 손상되었습니다.")
        else:
            st.error("먼저 메시지를 암호화하세요.")

with tab2:
    st.header("SHA-512 기반 무결성 포함 AES 암호복호화")
    with st.expander("이  $H(M) || M$  방식이란?"):
        st.markdown("""

```

이 방식은 메시지의 ****기밀성****과 ****무결성****을 동시에 제공하는 간단한 방법입니다.

- ****암호화****: 메시지(M)의 해시(H(M))를 계산한 뒤, 해시와 메시지를 합쳐(`H(M) || M`) AES로 암호화합니다.

- ****복호화 및 검증****: 암호문을 복호화하여 해시와 메시지를 분리합니다. 분리된 메시지로 다시 해시를 계산하여, 복호화된 해시와 일치하는지 비교합니다.

만약 두 해시 값이 일치하면 메시지가 전송 중에 변조되지 않았음을 확신할 수 있습니다.

```
"""
```

```
# 세션에 키 보관
```

```
if 'sha_aes_key' not in st.session_state:
```

```
    st.session_state['sha_aes_key'] = get_random_bytes(32) # AES-256
```

```
st.subheader("□ 키 관리 (AES-256)")
```

```
colk1, colk2 = st.columns([3, 1])
```

```
with colk1:
```

```
    st.code(base64.b64encode(st.session_state['sha_aes_key']).decode(), language='text')
```

```
with colk2:
```

```
    if st.button("□ 새 랜덤 키 생성"):
```

```
        st.session_state['sha_aes_key'] = get_random_bytes(32)
```

```
        st.success("새 AES-256 키가 생성되었습니다.")
```

```
sha_plain = st.text_area("암호화할 메시지 입력", key="sha_plain", placeholder="무결성을 검증할 메시지를 입력하세요.")
```

```
col1, col2 = st.columns(2)
```

```
with col1:
```

```
    if st.button("□ E(H(M)||M) 암호화", key="sha_integrity_enc"):
```

```
        if sha_plain:
```

```
            key = st.session_state['sha_aes_key']
```

```
            digest = hashlib.sha512(sha_plain.encode()).digest()
```

```
            payload = digest + sha_plain.encode()
```

```
            cipher = AES.new(key, AES.MODE_CBC)
```

```
            ct = cipher.encrypt(pad(payload, AES.block_size))
```

```
            iv = base64.b64encode(cipher.iv).decode()
```

```
            ct_b64 = base64.b64encode(ct).decode()
```

```
            st.session_state['sha_int_iv'] = iv
```

```
            st.session_state['sha_int_ct'] = ct_b64
```

```
            st.markdown("---")
```

```
            st.subheader("□ 암호화 및 무결성 보장 과정")
```

```
            st.markdown("**1. 메시지 해시 계산 (SHA-512)**")
```

```
                st.code(f"H(M) = SHA512('{sha_plain[:20]}...')\n=> {digest.hex()}",
```

```
language='text')
```

```

st.markdown("***2. 페이로드 결합: `H(M) || M`**")
st.markdown("계산된 64바이트 해시값을 원본 메시지 앞에 붙입니다.")
st.code(f"Payload = (64 bytes hash) + ({len(sha_plain.encode())} bytes message)", language='text')

st.markdown("***3. AES-CBC 암호화**")
st.markdown("결합된 페이로드를 `AES-256-CBC` 방식으로 암호화합니다.")
st.code(f"IV: {iv}\nCiphertext: {ct_b64}", language='text')
st.success("암호화 완료")
st.markdown("***IV:**")
st.code(iv, language='text')
st.markdown("***Ciphertext:**")
st.code(ct_b64, language='text')
else:
    st.error("메시지를 입력하세요.")

with col2:
    if st.button("□ 복호화 및 검증", key="sha_integrity_dec"):
        if 'sha_int_iv' in st.session_state and 'sha_int_ct' in st.session_state:
            try:
                key = st.session_state['sha_aes_key']
                iv = base64.b64decode(st.session_state['sha_int_iv'])
                ct = base64.b64decode(st.session_state['sha_int_ct'])
                cipher = AES.new(key, AES.MODE_CBC, iv)
                pt = unpad(cipher.decrypt(ct), AES.block_size)
                digest_rec = pt[:hashlib.sha512().digest_size]
                msg_rec = pt[hashlib.sha512().digest_size:]

                digest_calc = hashlib.sha512(msg_rec).digest()

                st.markdown("---")
                st.subheader("□ 복호화 및 무결성 검증 과정")
                st.markdown("***1. AES-CBC 복호화**")
                st.markdown("수신된 암호문을 키와 IV로 복호화하여 `H(M) || M` 페이로드를 복원합니다.")

                st.markdown("***2. 다이제스트 및 메시지 분리**")
                st.markdown("복호화된 페이로드에서 앞 64바이트(복원된 다이제스트)와 나머지(복원된 메시지)를 분리합니다.")

                st.markdown("***3. 다이제스트 재계산 및 비교**")
                st.markdown("복원된 메시지로 SHA-512 해시를 다시 계산하고, 1번 과정에서 복원된 다이제스트와 비교합니다.")

                st.subheader("◆ 검증 결과")
                st.text_area("복호화된 메시지", msg_rec.decode(), height=100)

```

```

        st.markdown("**복호화된 다이제스트:**")
        st.code(digest_rec.hex(), language='text')
        st.markdown("**계산된 다이제스트:**")
        st.code(digest_calc.hex(), language='text')

        if digest_rec == digest_calc:
            st.success("    무결성 검증 성공: 메시지가 변조되지 않았습니다.")
        else:
            st.error("    무결성 검증 실패: 메시지가 변경되었거나 잘못된 키입니다.")
    except Exception as e:
        st.error(f"복호화 오류: {e}")
    else:
        st.error("먼저 암호문을 생성하세요.")

```

with tab3:

```

    st.header("Alice → Bob 시뮬레이션")
    with st.expander(" i  시뮬레이션 설명"):
        st.markdown("""
        Alice가 선택한 암호화 알고리즘으로 메시지를 암호화하여 네트워크(중간자)로 전송하고,
        Bob이 수신하여 복호화하는 과정을 시뮬레이션합니다. 중간에서 공격자(Injection)가 암호문을 변
        조해
        Bob의 복호화 결과가 어떻게 되는지 확인할 수 있습니다.
        """)

```

키 및 알고리즘 설정

```

    st.subheader("□ 키 및 알고리즘 설정")
    colk1, colk2 = st.columns(2)
    with colk1:
        st.markdown("**RSA 키 쌍 (Alice & Bob)**")
        if 'alice_public' not in st.session_state or 'bob_public' not in st.session_state:
            if st.button("□ ALICE/BOB RSA 키 생성"):
                alice_public, alice_private = generate_rsa_keys()
                bob_public, bob_private = generate_rsa_keys()
                st.session_state['alice_public'] = alice_public
                st.session_state['alice_private'] = alice_private
                st.session_state['bob_public'] = bob_public
                st.session_state['bob_private'] = bob_private
                st.success("Alice와 Bob의 RSA 키 쌍이 생성되었습니다.")
        if 'bob_public' in st.session_state:
            st.text_area("BOB 공개키 (암호화용)", st.session_state['bob_public'], height=100)

    with colk2:
        st.markdown("**공유 대칭키 (AES용)**")
        if 'shared_aes_key' not in st.session_state:
            st.session_state['shared_aes_key'] = get_random_bytes(16)

```

```

if st.button("□ 새 대칭키 생성"):
    st.session_state['shared_aes_key'] = get_random_bytes(16)
    st.code(base64.b64encode(st.session_state['shared_aes_key']).decode(),
language='text')

algo = st.selectbox("암호화 알고리즘 선택", ["AES-CBC", "AES-OFB", "SHA512+AES(E(H||M))"],
index=0)

st.subheader("Alice: 암호화 및 전송")
alice_input = st.text_area("Alice가 보낼 메시지", key='alice_input', placeholder="Alice의
비밀 메시지...")

if st.button("□ Alice 암호화 & 전송"):
    if not alice_input:
        st.error("메시지를 입력하세요.")
    else:
        packet = {"alg": algo}
        # ... (기존 암호화 로직은 동일)
        if algo == "AES-CBC":
            key = st.session_state['shared_aes_key']
            cipher = AES.new(key, AES.MODE_CBC)
            ct = cipher.encrypt(pad(alice_input.encode(), AES.block_size))
            packet['iv'] = base64.b64encode(cipher.iv).decode()
            packet['ct'] = base64.b64encode(ct).decode()
            packet['explain'] = f"AES-CBC: IV={packet['iv'][:10]}..., 블록 사이즈=16, 패딩
PKCS7"

        elif algo == "AES-OFB":
            key = st.session_state['shared_aes_key']
            iv_b64, ct_b64 = aes_ofb_encrypt(alice_input, key)
            packet['iv'] = iv_b64
            packet['ct'] = ct_b64
            packet['explain'] = "AES-OFB: 스트림 방식, IV와 키로 키스트림 생성 후 XOR"

        elif algo == "SHA512+AES(E(H||M))":
            key = hashlib.sha512(st.session_state['shared_aes_key']).digest()[:32]
            digest = hashlib.sha512(alice_input.encode()).digest()
            payload = digest + alice_input.encode()
            cipher = AES.new(key, AES.MODE_CBC)
            ct = cipher.encrypt(pad(payload, AES.block_size))
            packet['iv'] = base64.b64encode(cipher.iv).decode()
            packet['ct'] = base64.b64encode(ct).decode()
            packet['explain'] = "E(H(M)||M): 메시지 앞에 SHA-512 다이제스트를 붙여 암호화. 복
호화 후 무결성 검증 가능"

        if packet:
            st.session_state['alice_packet'] = packet

```



```

        st.success("Alice가 암호문을 생성하여 전송했습니다. (중간자로 전달됨)")

# show on-wire packet
st.subheader("□ 네트워크(중간자) 패킷")
if 'alice_packet' in st.session_state:
    pkt = st.session_state['alice_packet'].copy()
    st.info(f"**알고리즘:** {pkt.get('alg')} | **설명:** {pkt.get('explain')}")
    st.json(pkt)

# Bob: 복호화
st.subheader("Bob: 수신 및 복호화")
if st.button("□ Bob 복호화 시도"):
    try:
        final_pkt = st.session_state['alice_packet']
        alg = final_pkt.get('alg')
        # ... (기존 복호화 로직은 동일)
        if alg == "AES-CBC":
            key = st.session_state['shared_aes_key']
            iv = base64.b64decode(final_pkt['iv'])
            ct = base64.b64decode(final_pkt['ct'])
            cipher = AES.new(key, AES.MODE_CBC, iv)
            pt = unpad(cipher.decrypt(ct), AES.block_size)
            st.success("복호화 성공")
            st.code(pt.decode())
        elif alg == "AES-OFB":
            key = st.session_state['shared_aes_key']
            pt = aes_ofb_decrypt(final_pkt['iv'], final_pkt['ct'], key)
            st.success("복호화 성공")
            st.code(pt)
        elif alg == "SHA512+AES(E(H|M))":
            key = hashlib.sha512(st.session_state['shared_aes_key']).digest()[:32]
            iv = base64.b64decode(final_pkt['iv'])
            ct = base64.b64decode(final_pkt['ct'])
            cipher = AES.new(key, AES.MODE_CBC, iv)
            pt = unpad(cipher.decrypt(ct), AES.block_size)
            digest_rec = pt[:hashlib.sha512().digest_size]
            msg_rec = pt[hashlib.sha512().digest_size:]
            digest_calc = hashlib.sha512(msg_rec).digest()
            st.code(f"복호화된 메시지: {msg_rec.decode()}\n복호화된 다이제스트: {digest_rec.hex()}\n계산한 다이제스트: {digest_calc.hex()}")
            if digest_rec == digest_calc:
                st.success("무결성 검증 성공")
            else:
                st.error("무결성 검증 실패: 메시지가 변조되었거나 잘못된 키")
    except Exception as e:

```

```
st.error(f"복호화 중 치명적 오류 발생: {e}")
```

```
with tab4:
```

```
    st.header("고전 암호 시스템")
```

```
    with st.expander("고전 암호 시스템 소개"):
```

```
        st.markdown("""
```

1. ****카이사르 암호 (Caesar Cipher)****: 각 문자를 일정한 수만큼 알파벳 상에서 이동시키는 가장 간단한 형태의 치환 암호입니다.

2. ****아핀 암호 (Affine Cipher)****: 일차 함수($(x) = ax + b \pmod{m}$)를 사용한 치환 암호로, 카이사르보다 복잡한 치환을 제공합니다.

3. ****비즈네르 암호 (Vigenere Cipher)****: 여러 개의 카이사르 암호를 키워드를 이용해 반복적으로 적용하는 다중 치환 암호입니다.

4. ****조합 암호****: 카이사르 암호와 아핀 암호를 순차적으로 적용하여 다중 암호화로 보안성을 향상시킵니다.

```
        """)
```

```
    cipher_type = st.radio(
```

```
        "암호화 방식 선택",
```

```
        ["카이사르 암호", "아핀 암호", "비즈네르 암호", "조합 암호"],
```

```
        horizontal=True,
```

```
        help="사용할 고전 암호 방식을 선택하세요"
```

```
    )
```

```
    text = st.text_area("암호화/복호화할 텍스트 입력", key="classic_text", placeholder="영문 텍스트를 입력하세요...")
```

```
    if cipher_type == "카이사르 암호":
```

```
        shift = st.slider("시프트 값", min_value=1, max_value=25, value=3)
```

```
        col1, col2 = st.columns(2)
```

```
        with col1:
```

```
            if st.button("카이사르 암호화", key="caesar_enc"):
```

```
                if text:
```

```
                    encrypted = caesar_encrypt(text, shift)
```

```
                    st.session_state['caesar_result'] = encrypted
```

```
                    st.markdown("---")
```

```
                    # --- AI 활용: 수학 공식 표현 ---
```

```
                    # 암호화 원리를 명확하게 전달하기 위해, 카이사르 암호의 수학적 표현을
```

```
                    # Streamlit의 st.latex 기능을 사용하여 수식으로 표현하는 방법을 AI의 도움을 받아 구현했습니다.
```

```
                    st.subheader("카이사르 암호화 과정")
```

```
                    st.markdown(f"각 알파벳 문자를 알파벳 순서상 **{shift}칸** 뒤로 이동시킵니다.")
```

```
                    example_char = next((c for c in text if c.isalpha()), 'A')
```

```
                    encrypted_char = caesar_encrypt(example_char, shift)
```

```

st.latex(f"E(x) = (x + {shift}) \pmod{{26}}")
st.markdown(f"**예시** : `{example_char}` → `{encrypted_char}`")

st.text_area("암호화 결과", encrypted, height=100)

with col2:
    if st.button("□ 카이사르 복호화", key="caesar_dec"):
        if 'caesar_result' in st.session_state:
            decrypted = caesar_decrypt(st.session_state['caesar_result'], shift)
            st.text_area("복호화 결과", decrypted, height=100)

st.markdown("---")
st.subheader("□ 카이사르 암호 해독 (Brute-Force with Chi-Square)")
if st.button("□ 카이제곱 통계로 암호 해독", help="암호화된 텍스트에 대해 모든 키를 시도하
고, 통계적으로 가장 그럴듯한 평문을 찾습니다."):
    if 'caesar_result' in st.session_state and st.session_state['caesar_result']:
        ciphertext = st.session_state['caesar_result']
        scores = []
        for k in range(26):
            decrypted = caesar_decrypt(ciphertext, k)
            score = chi_square_score(decrypted)
            scores.append((score, k, decrypted))

        scores.sort(key=lambda x: x[0]) # 점수가 낮은 순으로 정렬
        st.markdown("**가장 가능성 높은 복호화 결과 (상위 5개):**")
        for score, key, result in scores[:5]:
            st.info(f"**Key: {key}** (Score: {score:.2f})\n> {result}")

elif cipher_type == "아핀 암호":
    col1, col2 = st.columns(2)
    with col1:
        a = st.number_input("a 값 (26과 서로소여야 함)", min_value=1, max_value=25, value=5,
step=2)
    with col2:
        b = st.number_input("b 값", min_value=0, max_value=25, value=8)

    if gcd(a, 26) != 1:
        st.error(f"a={a}는 26과 서로소가 아닙니다. 다른 값을 선택하세요. (예: 1, 3, 5, 7, 9,
11, 15, 17, 19, 21, 23, 25)")
    else:
        col1, col2 = st.columns(2)
        with col1:
            if st.button("□ 아핀 암호화", key="affine_enc"):
                if text:

```

```

        encrypted = affine_encrypt(text, a, b)
        st.session_state['affine_result'] = encrypted
        st.markdown("---")
        st.subheader("□ 암호화 함수")
        st.latex(f"E(x) = ({a}x + {b}) \pmod{{26}}")
        example_char = next((c for c in text if c.isalpha()), 'A')
        encrypted_char = affine_encrypt(example_char, a, b)
        st.markdown(f"**예시**: `{example_char}` → `{encrypted_char}`")
        st.text_area("암호화 결과", encrypted, height=100)

    with col2:
        if st.button("□ 아핀 복호화", key="affine_dec"):
            if 'affine_result' in st.session_state:
                st.markdown("---")
                st.subheader("□ 복호화 함수")
                a_inv = mod_inverse(a, 26)
                st.latex(f"D(y) = {a_inv}(y - {b}) \pmod{{26}}")
                decrypted = affine_decrypt(st.session_state['affine_result'], a, b)
                st.text_area("복호화 결과", decrypted, height=100)

    elif cipher_type == "비즈네르 암호":
        vigenere_key = st.text_input("비즈네르 키 (영문)", key="vigenere_classic_key",
placeholder="e.g., 'KEY'")

    col1, col2 = st.columns(2)
    with col1:
        if st.button("□ 비즈네르 암호화", key="vigenere_enc"):
            if text and vigenere_key:
                try:
                    encrypted = vigenere_encrypt(text, vigenere_key)
                    st.session_state['vigenere_result'] = encrypted
                    st.markdown("---")
                    st.subheader("□ 암호화 과정")
                    st.markdown(f"평문 `{text[:10]}...`을 키 `{vigenere_key}`를 반복 사
용하여 각 문자마다 다른 시저 암호(이동값)를 적용합니다.")
                    st.latex(f"E_i(P_i) = (P_i + K_i) \pmod{{26}}")

                    st.text_area("암호화 결과", encrypted, height=100)
                except ValueError as e:
                    st.error(str(e))
            else:
                st.warning("텍스트와 키를 모두 입력하세요.")
    with col2:
        if st.button("□ 비즈네르 복호화", key="vigenere_dec"):
            if 'vigenere_result' in st.session_state and vigenere_key:

```



```

        st.markdown("**1단계: 아핀 복호화**")
        a_inv = mod_inverse(affine_a, 26)
        st.latex(f"A^{{-1}}(y) = {a_inv}(y - {affine_b}) \pmod{{26}}")
        st.markdown("**2단계: 카이사르 복호화**")
        st.latex(f"C^{{-1}}(z) = (z - {caesar_shift}) \pmod{{26}}")
        st.text_area("복호화 결과", decrypted, height=100)

# ----- AES-OFB 탭 UI -----
with tab5:
    st.header("AES-OFB 대칭키 암호화")
    with st.expander("이 OFB 모드란?"):
        st.markdown("""
            OFB(Output Feedback) 모드는 블록 암호를 스트림 암호처럼 사용하게 해주는 동작 모드입니다.
            - **암호화**: IV를 키로 암호화하여 첫 키 스트림 블록을 생성하고, 이를 평문 블록과 XOR하여 암호문을 만듭니다. 다음 키 스트림은 이전 **키 스트림 블록**을 다시 암호화하여 생성합니다.
            - **복호화**: 암호화와 동일한 과정으로 키 스트림을 생성하여 암호문과 XOR합니다.
            - **장점**: 암호문 한 비트의 오류가 평문의 해당 비트에만 영향을 미치므로 오류 전파가 적습니다.
            """)

    # 세션에 AES-OFB 키 보관
    if 'ofb_key' not in st.session_state:
        st.session_state['ofb_key'] = get_random_bytes(16)

    st.subheader("키 관리 (AES-128)")
    colk1, colk2 = st.columns([3,1])
    with colk1:
        st.code(base64.b64encode(st.session_state['ofb_key']).decode(), language='text')
    with colk2:
        if st.button("새 OFB 키 생성"):
            st.session_state['ofb_key'] = get_random_bytes(16)
            st.success("새 AES-OFB 키가 생성되었습니다.")

    ofb_plain = st.text_area("평문 입력", key="ofb_plain", placeholder="스트림 암호화할 메시지를 입력하세요.")
    c1, c2 = st.columns(2)
    with c1:
        if st.button("OFB 암호화", key="ofb_encrypt_btn"):
            if ofb_plain:
                iv_b64, ct_b64 = aes_ofb_encrypt(ofb_plain, st.session_state['ofb_key'])
                st.session_state['ofb_iv'] = iv_b64
                st.session_state['ofb_ct'] = ct_b64
                st.markdown("---")
                st.subheader("암/복호화 과정 (XOR)")
                st.markdown("""

```

```

1.  **키 스트림 생성**: `AES(Key, IV)`를 계산하여 첫 키 스트림 블록을 생성합니다.
2.  **XOR 연산**: 생성된 키 스트림과 평문을 XOR하여 암호문을 생성합니다.
`Ciphertext = Plaintext ⊕ Keystream`
3.  복호화는 암호문에 동일한 키 스트림을 다시 XOR하여 평문을 복원합니다. `Plaintext
= Ciphertext ⊕ Keystream`
    """
    st.markdown("**IV (Base64)**")
    st.code(iv_b64, language='text')
    st.markdown("**Ciphertext (Base64)**")
    st.code(ct_b64, language='text')
else:
    st.error("평문을 입력하세요.")

with c2:
    if st.button("❏ OFB 복호화", key="ofb_decrypt_btn"):
        if 'ofb_iv' in st.session_state and 'ofb_ct' in st.session_state:
            try:
                pt = aes_ofb_decrypt(st.session_state['ofb_iv'],
st.session_state['ofb_ct'], st.session_state['ofb_key'])
                st.markdown("**복호화된 평문**")
                st.code(pt, language='text')
            except Exception as e:
                st.error(f"복호화 실패: {e}")
        else:
            st.error("먼저 암호문을 생성하세요.")

# ----- RSA 디지털 서명 탭 UI -----
with tab6:
    st.header("RSA 기반 디지털 서명")
    with st.expander("❏ 디지털 서명이란?"):
        st.markdown("""
            디지털 서명은 메시지의 **무결성(Integrity)**과 **인증(Authentication)**, **부인 방지(Non-repudiation)**를 보장하는 핵심 기술입니다.
            - **서명 생성**: 송신자는 자신의 **개인키**로 메시지의 해시(SHA-512)를 암호화하여 서명을 생성합니다.
            - **서명 검증**: 수신자는 송신자의 **공개키**로 서명을 복호화하고, 직접 계산한 메시지 해시와 비교하여 서명을 검증합니다.

            개인키는 소유자만 가지고 있으므로, 공개키로 검증에 성공했다는 것은 해당 개인키 소유자가 서명했음을 증명합니다.
        """)

    # 키 생성 및 관리
    st.subheader("❏ RSA 키 쌍 관리 (서명용)")
    if 'sig_private_key' not in st.session_state:
        if st.button("❏ 새 RSA 키 쌍 생성 (서명용)"):

```

```

pub, priv = generate_rsa_keys()
st.session_state['sig_public_key'] = pub
st.session_state['sig_private_key'] = priv
st.success("새로운 2048비트 RSA 키 쌍이 생성되었습니다.")

if 'sig_private_key' in st.session_state:
    col1, col2 = st.columns(2)
    with col1:
        st.text_area("개인키 (서명용)", st.session_state['sig_private_key'], height=150)
    with col2:
        st.text_area("공개키 (검증용)", st.session_state['sig_public_key'], height=150)

# 서명 생성 및 검증
sig_message = st.text_area("서명할 메시지", key="sig_message", placeholder="서명을 생성할 원
본 메시지를 입력하세요.")
if st.button("🔑 서명 생성 및 검증 실행"):
    if sig_message and 'sig_private_key' in st.session_state:
        # 서명 생성
        signature = rsa_sign(sig_message.encode(), st.session_state['sig_private_key'])
        st.markdown("---")
        st.subheader("🔑 서명 생성 과정")
        st.markdown("""
1. **메시지 해시**: 원본 메시지에 대해 SHA-512 해시 함수를 적용하여 64바이트 다이제스트
를 생성합니다.
2. **개인키로 암호화**: 생성된 다이제스트를 송신자의 **개인키**로 암호화(서명)합니다.
이 결과가 바로 디지털 서명입니다.

이 서명은 오직 해당 개인키의 소유자만이 생성할 수 있습니다.
""")

        st.markdown("**생성된 서명 (Base64)**")
        st.code(base64.b64encode(signature).decode(), language='text')

# 즉시 검증
is_valid = rsa_verify(sig_message.encode(), signature,
st.session_state['sig_public_key'])
st.markdown("**자체 검증 결과**")
if is_valid:
    st.markdown("---")
    st.subheader("🔍 서명 검증 과정")
    st.markdown("""
1. **서명 복호화**: 수신된 서명을 송신자의 **공개키**로 복호화하여 원본 다이제스트
(`H1`)를 얻습니다.
2. **메시지 재해시**: 함께 수신된 원본 메시지에 대해 동일한 SHA-512 해시 함수를 적
용하여 새로운 다이제스트(`H2`)를 계산합니다.

```



```
3.  **비교**: `H1`과 `H2`가 일치하는지 확인합니다. 일치하면 서명이 유효한 것입니다.  
    """)  
  
    st.success("    서명이 유효합니다. (메시지 무결성 및 출처 확인)")  
else:  
    st.error("    서명이 유효하지 않습니다.")  
else:  
    st.warning("메시지를 입력하고 키를 먼저 생성하세요.")
```

2. 주요 기능 및 구현 내용

2.1. 사용자 인터페이스 (UI/UX)

직관적인 다크 모드(Dark Mode) 인터페이스를 적용했습니다. 각 기능은 탭(Tab)으로 명확하게 구분하였고 동작 과정은 확장으로 제시하였습니다. 암호화에 대한 설명을 제시하였습니다.

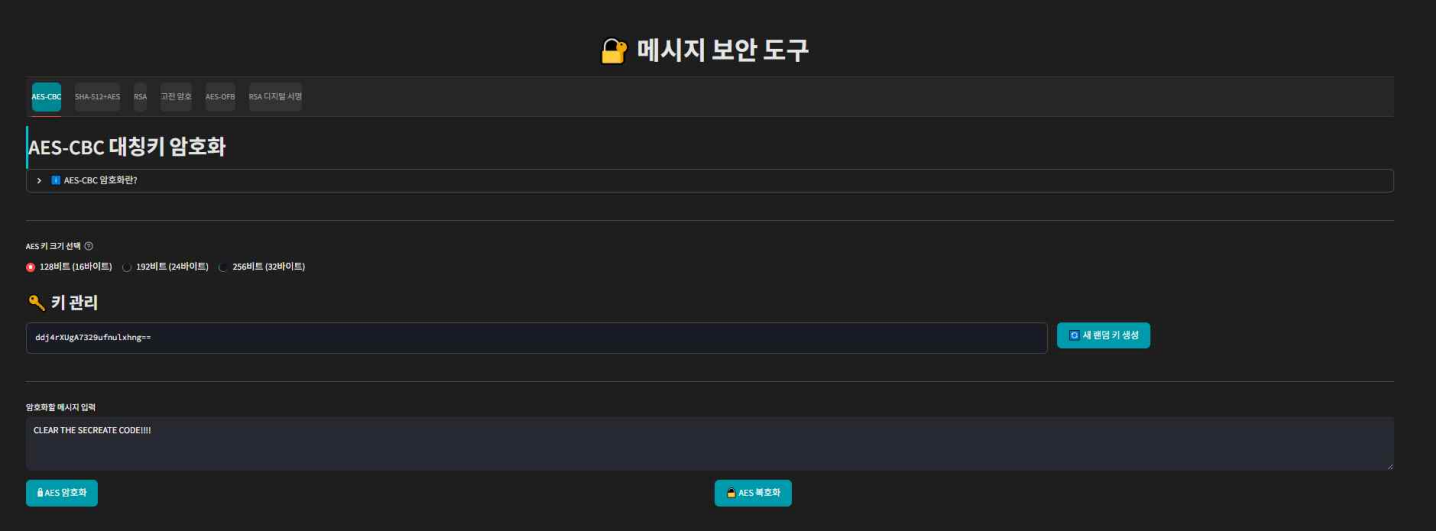
2.2. 대칭키 암호 (Symmetric-Key Ciphers)

1) AES-CBC (Cipher Block Chaining)

구현: 사용자가 키 크기를 (128bit, 192bit, 256bit)로 선택 할 수 있습니다. 새 랜덤 키 생성으로 대칭키를 생성 합니다. 그리고 전송하고자하는 메시지를 입력합니다. 암호화를 누르면 AES의 과정과 암호화 결과를 보여줍니다. AES 복호화를 누르면 과정하고 메시지를 보여줍니다.

특징: 암호화 시 PKCS7 패딩, IV(초기화 벡터) 생성, 암호화 블록 체이닝 과정을 단계별로 설명하여 CBC 모드의 동작 원리를 시각적으로 보여줍니다.

[결과]



ddj4rXUgA7329ufnuLxhg==

새 랜덤 키 생성

암호화할 메시지 입력

CLEAR THE SECREATE CODE!!!!

AES 암호화

AES 복호화

암호화 과정

1. 초기화 벡터 (IV) 생성

d5Ypt5bFhEJ8j57Mz8UhxA==

CBC 모드의 첫 블록 암호화에 사용될 무작위 16바이트 값입니다.

2. PKCS7 패딩

평문을 16바이트 블록 단위로 맞추기 위해 패딩을 추가합니다.

원본 길이 : 27 바이트
패딩 후 길이 : 32 바이트

3. 암호화 결과

패딩된 평문이 IV와 함께 AES-CBC 모드로 암호화된 후, IV와 암호문이 각각 Base64로 인코딩됩니다.

IV: d5Ypt5bFhEJ8j57Mz8UhxA==
Ciphertext: 40DzZ3bZQm9+3Mec1acZ2LvZ99QAk1PatQ8/6ZknkMQ=

암호화 결과

초기화 벡터 (IV):

d5Ypt5bFhEJ8j57Mz8UhxA==

암호문 (Base64):

40DzZ3bZQm9+3Mec1acZ2LvZ99QAk1PatQ8/6ZknkMQ=

AES 키 크기 선택

128비트 (16바이트) 192비트 (24바이트) 256비트 (32바이트)

키 관리

ddj4rXUgA7329ufnuLxhg==

새 랜덤 키 생성

암호화할 메시지 입력

CLEAR THE SECREATE CODE!!!!

AES 암호화

AES 복호화

복호화 결과

복호화된 메시지

CLEAR THE SECREATE CODE!!!!

복호화 과정

- Base64 디코딩: Base64로 인코딩된 IV와 암호문을 다시 바이트 형태로 변환합니다.
- AES-CBC 복호화: 입력된 키와 디코딩된 IV를 사용하여 암호문을 복호화합니다.
- 패딩 제거: 복호화된 데이터의 끝에 추가되었던 PKCS7 패딩을 제거하여 원본 메시지를 복원합니다.

복호화 성공! 원본 메시지가 정확히 복원되었습니다.

2) AES-OFB (Output Feedback)

구현: 블록 암호인 AES를 스트림 암호처럼 사용하는 OFB 모드를 합니다.

특징: 키를 랜덤으로 새로 생성하고 IV로 키 스트림(Keystream)을 생성하고, 이를 평문과 XOR하는 과정을 설명하여 스트림 암호의 핵심 원리를 제시합니다.

[결과]

AES-CBCSHA-512+AESRSAG고전 암호AES-OFBRSA 디지털 서명

AES-OFB 대칭키 암호화

OFB 모드란?

OFB(Output Feedback) 모드는 블록 암호를 스트림 암호처럼 사용하게 해주는 동작 모드입니다.

- 암호화: IV를 키로 암호화하여 첫 키 스트림 블록을 생성하고, 이를 평문 블록과 XOR하여 암호문을 만듭니다. 다음 키 스트림은 이전 키 스트림 블록을 다시 암호화하여 생성합니다.
- 복호화: 암호화와 동일한 과정으로 키 스트림을 생성하여 암호문과 XOR합니다.
- 장점: 암호문 한 비트의 오류가 평문의 해당 비트에만 영향을 미치므로 오류 전파가 적습니다.

키 관리 (AES-128)

KaQbEQGzC8OMc76Lsv7+1Q==

새 OFB 키 생성

평문 입력

CLEAR THE SECREATE CODE!!!!

OFB 암호화

OFB 복호화

암/복호화 과정 (XOR)

- 키 스트림 생성: $AES(Key, IV)$ 를 계산하여 첫 키 스트림 블록을 생성합니다.
- XOR 연산: 생성된 키 스트림과 평문을 XOR하여 암호문을 생성합니다. $Ciphertext = Plaintext \oplus Keystream$
- 복호화는 암호문에 동일한 키 스트림을 다시 XOR하여 평문을 복원합니다. $Plaintext = Ciphertext \oplus Keystream$

IV (Base64)

dxu+w30CHqbGWIS6EtR4Aw==

Ciphertext (Base64)

ZQ24MBA/AQbz2UejXtiEriF0YY7fYyrs2RAV

OFB 복호화

복호화된 평문

CLEAR THE SECREATE CODE!!!!

2.3. 무결성 보장 암호 (Authenticated Encryption)

SHA-512 + AES

구현: 메시지의 기밀성(Confidentiality)과 무결성(Integrity)을 동시에 제공하는 $E(H(M) \parallel M)$ 방식을 구현했다. 사용자가 비밀번호를 입력하는 대신, 보안성이 높은 무작위 대칭키를 생성하여 사용하도록 변경합니다.

암호화 과정:

원본 메시지(M)에 대해 SHA-512 해시값 $H(M)$ 을 계산합니다.

$H(M)$ 과 M을 결합($H(M) \parallel M$)하여 페이로드를 생성합니다.

생성된 페이로드를 AES-CBC 모드로 암호화합니다.

복호화 및 검증 과정:

암호문을 복호화하여 $H(M)$ 과 M을 분리합니다.

분리된 M으로 다시 SHA-512 해시값 $H'(M)$ 을 계산합니다.

복호화된 $H(M)$ 과 새로 계산한 $H'(M)$ 을 비교하여 메시지 변조 여부를 확인합니다.

[결과]

AES-CBCSHA-512+AESRSAGOSTN 암호AES-OFBRSA 디지털 서명

SHA-512 기반 무결성 포함 AES 암호화

E(H(M) || M) 방식이란?

이 방식은 메시지의 기밀성과 무결성을 동시에 제공하는 간단한 방법입니다.

• 암호화: 메시지(M)의 해시(H(M))를 계산한 뒤, 해시와 메시지를 합쳐($H(M) \parallel M$) AES로 암호화합니다.

• 복호화 및 검증: 암호문을 복호화하여 해시와 메시지를 분리합니다. 분리된 메시지로 다시 해시를 계산하여, 복호화된 해시와 일치하는지 비교합니다.

만약 두 해시 값이 일치하면 메시지가 전송 중에 변조되지 않았음을 확신할 수 있습니다.

키 관리 (AES-256)

I8eno5g8f057uJr9Vi8EjecryC+K4hiQggvRT1VBqMM=

새 랜덤 키 생성

암호화할 메시지 입력

CLEAR THE SECRETE CODE!!!!

E(H(M)||M) 암호화

복호화 및 검증

암호화 및 무결성 보장 과정

1. 메시지 해시 계산 (SHA-512)

```
H(M) = SHA512('CLEAR THE SECRETE C...')  
=> 55660065360b71bdd37bb821c5434248f7149c06373a6328ded4d8f9c97c2491e506bc1f7a74f877232bf4cb635c87006a
```

2. 페이로드 결합: $H(M) || M$

계산된 64바이트 해시값을 원본 메시지 앞에 붙입니다.

```
Payload = (64 bytes hash) + (27 bytes message)
```

3. AES-CBC 암호화

결합된 페이로드를 AES-256-CBC 방식으로 암호화합니다.

```
IV: aWJa6IZgPGlclKAlDLVWBg==  
Ciphertext: wFVJ4FfchKNQKyG7fcYJZw/o39if81huewD0yw+8pYj7FgHKKrR8JVGB40DuE3wCLpW6Iy/yiPUq8HxT6poHTdsL2
```

암호화 완료

IV:

```
aWJa6IZgPGlclKAlDLVWBg==
```

Ciphertext:

```
wFVJ4FfchKNQKyG7fcYJZw/o39if81huewD0yw+8pYj7FgHKKrR8JVGB40DuE3wCLpW6Iy/yiPUq8HxT6poHTdsL2TtqlI2xNaYzi
```

복호화 및 무결성 검증 과정 ↔

1. AES-CBC 복호화

수신된 암호문을 키와 IV로 복호화하여 $H(M) || M$ 페이로드를 복원합니다.

2. 다이제스트 및 메시지 분리

복호화된 페이로드에서 앞 64바이트(복원된 다이제스트)와 나머지(복원된 메시지)를 분리합니다.

3. 다이제스트 재계산 및 비교

복원된 메시지로 SHA-512 해시를 다시 계산하고, 1번 과정에서 복원된 다이제스트와 비교합니다.

검증 결과

복호화된 메시지

```
CLEAR THE SECRETE CODE!!!!
```

복호화된 다이제스트:

```
55660065360b71bdd37bb821c5434248f7149c06373a6328ded4d8f9c97c2491e506bc1f7a74f877232bf4cb635c87006accd
```

계산된 다이제스트:

```
55660065360b71bdd37bb821c5434248f7149c06373a6328ded4d8f9c97c2491e506bc1f7a74f877232bf4cb635c87006accd
```

☒ 무결성 검증 성공: 메시지가 변조되지 않았습니다.

2.4. 공개키 암호 (Public-Key Ciphers)

1) RSA 암호/복호화

구현: Alice → Bob 통신 시뮬레이션 탭에서 RSA 공개키 암호화를 구현했다. Alice는 Bob의 공개키로 메시지를 암호화하며, 이 암호문은 Bob의 개인키로만 복호화할 수 있다. 암호화 알고리즘을 AES_OFB,AES_CBC,SHA-512를 선택할 수 있다. 중간 패킷에 대한 정보를 보여주고 그것을 바탕으로 복호화를 진행합니다.

KEY 생성: 랜덤으로 RSA키를 생성합니다
암호화 알고리즘을 선택: AES-CBC, AES-OFB, SHA-512 중 하나를 선택하고 랜덤으로 대칭키를 생성합니다
보낼 메시지를 입력합니다.
보내는 과정을 네트워크 패킷으로 JSON으로 보여줍니다.
bob이 복호화합니다.

Alice → Bob 시뮬레이션

> 시뮬레이션 설정

키 및 알고리즘 설정

RSA 키 쌍 (Alice & Bob)

공유 대칭키 (AES용)

ALICE/BOB RSA 키 생성

새 대칭키 생성

uNesK0080AFqxIh0q6MoLQ==

암호화 알고리즘 선택

AES-CBC

Alice: 암호화 및 전송

Alice가 보낼 메시지

Alice의 비밀 메시지...

Alice 암호화 & 전송

Alice: 암호화 및 전송

Alice가 보낼 메시지

CLEAR THE SECRETE CODE!!!!

Alice 암호화 & 전송

네트워크(중간자) 패킷

알고리즘: AES-CBC | 설명: AES-CBC: IV=orTyaxJ6v/..., 블록 사이즈=16, 패딩 PKCS7

```
{
  "alg": "AES-CBC",
  "iv": "orTyaxJ6v/VRTghobTjCw==",
  "ct": "3X81P8htt1m99S0mCmu5Q==",
  "explain": "AES-CBC: IV=orTyaxJ6v/..., 블록 사이즈=16, 패딩 PKCS7"
}
```

Bob: 수신 및 복호화

Bob 복호화 시도

2) RSA 디지털 서명

구현: 메시지의 무결성, 인증, 부인 방지를 보장하는 RSA 디지털 서명 기능을 별도의 탭으로 구현했습니다.

서명 과정: 송신자가 자신의 개인키를 사용하여 메시지의 SHA-512 해시값을 암호화(서명)합니다.

검증 과정: 수신자가 송신자의 공개키를 사용하여 서명을 복호화하고, 함께 전송된 메시지의 해시값과 비교하여 서명의 유효성을 검증합니다.

RSA 기반 디지털 서명

디지털 서명이란?

디지털 서명은 메시지의 **무결성(Integrity)**과 **인증(Authentication)**, **부인 방지(Non-repudiation)**를 보장하는 핵심 기술입니다.

- 서명 생성:** 송신자는 자신의 **개인키**로 메시지의 해시(SHA-512)를 암호화하여 서명을 생성합니다.
- 서명 검증:** 수신자는 송신자의 **공개키**로 서명을 복호화하고, 직접 계산한 메시지 해시와 비교하여 서명을 검증합니다.

개인키는 소유자만 가지고 있으므로, 공개키로 검증에 성공했다는 것은 해당 개인키 소유자가 서명했음을 증명합니다.

RSA 키 쌍 관리 (서명용)

새 RSA 키 쌍 생성 (서명용)

서명할 메시지

서명을 생성할 원본 메시지를 입력하세요.

서명 생성 및 검증 실행

RSA 기반 디지털 서명

디지털 서명이란?

디지털 서명은 메시지의 **무결성(Integrity)**과 **인증(Authentication)**, **부인 방지(Non-repudiation)**를 보장하는 핵심 기술입니다.

- 서명 생성:** 송신자는 자신의 **개인키**로 메시지의 해시(SHA-512)를 암호화하여 서명을 생성합니다.
- 서명 검증:** 수신자는 송신자의 **공개키**로 서명을 복호화하고, 직접 계산한 메시지 해시와 비교하여 서명을 검증합니다.

개인키는 소유자만 가지고 있으므로, 공개키로 검증에 성공했다는 것은 해당 개인키 소유자가 서명했음을 증명합니다.

RSA 키 쌍 관리 (서명용)

개인키 (서명용)

공개키 (검증용)

-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAA3MwKQr/7tUzgJyhxYN3bd0xWMMEuDA8LOALWoxfFwkF0n2+U
qcFpf6+n048Vx23UJMSCo6PLI47bJb7vzQTYHtJaM23ne9D2ni8ATvvn4cNyluxF
QC6zD+9zY4GXTJLs+bpwNTgZ/IC7zvcTTUjeburQckpZK1GMkPGKZl9oBgJfMFep
/Vwo2Bwpx/GU04LRtCUlF+oyIKr/fzrAUKGOTUmTR9GGI8RpyrcQo40bCNeOLVe

-----BEGIN PUBLIC KEY-----
MIIBjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAA3MwKQr/7tUzgJyhxYN3b
d0xWMMEuDA8LOALWoxfFwkF0n2+UqcFpf6+n048Vx23UJMSCo6PLI47bJb7vzQTY
HtJaM23ne9D2ni8ATvvn4cNyluxFQC6zD+9zY4GXTJLs+bpwNTgZ/IC7zvcTTUje
burQckpZK1GMkPGKZl9oBgJfMFep/Vwo2Bwpx/GU04LRtCUlF+oyIKr/fzrAUKGO

서명할 메시지

CLEAR THE SECRETE CODE!!!!

서명 생성 및 검증 실행

2.5. 고전 암호 (Classical Ciphers)

단순 치환 암호부터 다중 치환 암호까지 다양한 고전 암호를 구현하여 암호학의 기본 원리를 학습할 수 있도록 구성했습니다.

카이사르(Caesar) 암호: 알파벳을 일정한 거리만큼 밀어서 치환. 카이제곱을 통해 복호화합니다.

아핀(Affine) 암호: 일차함수 $(ax + b) \bmod m$ 을 이용한 치환 암호.

비즈네르(Vigenere) 암호: 키워드를 반복 사용하여 여러 개의 카이사르 암호를 적용하는 다중 치환 암호.

조합 암호: 카이사르 암호와 아핀 암호를 순차적으로 적용하여 보안성을 높인 방식.

🔥 서명 생성 과정

- 메시지 해시:** 원본 메시지에 대해 SHA-512 해시 함수를 적용하여 64바이트 다이제스트를 생성합니다.
- 개인키로 암호화:** 생성된 다이제스트를 송신자의 **개인키**로 암호화(서명)합니다. 이 결과가 바로 디지털 서명입니다.

이 서명은 오직 해당 개인키의 소유자만이 생성할 수 있습니다.

생성된 서명 (Base64)

```
vqv0Rv1DJ04jb2RU/rG/YM9JUXdLhP9opCohASVC6Y92sVmJj8mQLVMsoYEWcsttnhIyTKu1Ylhl18vfDpn/f581sT3CR0rNAa34qrY1FAK1JgbUo3FvQ5KBM/QzNUMqH8oNtFn3Z662C2w9wzy5tg18NNmS1Fn71bw8CtZSm4UkeXXyV4/00aVHs2bJNqVwN18yQ9zdkKe6
```

자체 검증 결과

🔍 서명 검증 과정

- 서명 복호화:** 수신된 서명을 송신자의 **공개키**로 복호화하여 원본 다이제스트(H_1)를 얻습니다.
- 메시지 재해시:** 함께 수신된 원본 메시지에 대해 동일한 SHA-512 해시 함수를 적용하여 새로운 다이제스트(H_2)를 계산합니다.
- 비교:** H_1 과 H_2 가 일치하는지 확인합니다. 일치하면 서명이 유효한 것입니다.

✅ 서명이 유효합니다. (메시지 무결성 및 출처 확인)

고전 암호 시스템

📖 고전 암호 시스템 소개

- 카이사르 암호 (Caesar Cipher):** 각 문자를 일정한 수만큼 알파벳 상에서 이동시키는 가장 간단한 형태의 치환 암호입니다.
- 아핀 암호 (Affine Cipher):** 일차 함수 $(x) = ax + b \bmod(m)$ 을 사용한 치환 암호로, 카이사르보다 복잡한 치환을 제공합니다.
- 비즈네르 암호 (Vigenere Cipher):** 여러 개의 카이사르 암호를 키워드를 이용해 반복적으로 적용하는 다중 치환 암호입니다.
- 조합 암호:** 카이사르 암호와 아핀 암호를 순차적으로 적용하여 다중 암호화로 보안성을 향상시킵니다.

암호화 방식 선택

☒ 카이사르 암호 ☐ 아핀 암호 ☐ 비즈네르 암호 ☐ 조합 암호

암호화/복호화할 텍스트 입력

CLEAR THE SECRETE CODE!!!!

시프트 값

4

🔒 카이사르 암호화

🔓 카이사르 복호화

🔥 카이사르 암호 해독 (Brute-Force with Chi-Square)

🔑 카이제곱 통계로 암호 해독

암호화 과정 ↗

각 알파벳 문자를 알파벳 순서상 4칸 뒤로 이동시킵니다.

$$E(x) = (x + 4) \pmod{26}$$

예시: C → G

암호화 결과

GPIEV XLI WIGVIXI GSHI!!!!

🔥 카이사르 암호 해독 (Brute-Force with Chi-Square)

🔒 카이제곱 통계로 암호 해독

가장 가능성 높은 복호화 결과 (상위 5개):

Key: 4 (Score: 23.04)

CLEAR THE SECREATE CODE!!!!

Key: 15 (Score: 36.07)

RATPG IWT HTRGTPIT RDST!!!!

Key: 1 (Score: 65.74)

FOH DU WKH VHFUHDWH FRGH!!!!

Key: 20 (Score: 77.71)

MVOKB DRO COMBOKDO MYNO!!!!

Key: 3 (Score: 101.52)

DMFBS UIF TFDSFBUF DPEFI!!!!

[AI 사용부분 설명]

- AI는 전체적인 색상 조합, 폰트, 버튼, 탭 스타일 등 UI 디자인을 위한 CSS 코드 생성에 활용되었다.
- 사용자 경험을 개선하기 위해 각 기능에 맞는 아이콘(, ,  등)을 추천받아 버튼에 적용하였다.
- 코드 리팩토링 및 구조화:
기존에 분리되어 있던 암호화 기능들을 하나의 Streamlit 애플리케이션으로 통합하고, 탭(Tab) 설명구조를 구성하는 과정에서 코드 구조에 대한 조언을 얻었습니다.
- 각 암호화 알고리즘의 동작 원리를 사용자가 쉽게 이해할 수 있도록, 단계별 설명 텍스트와 `st.latex`를 이용한 수학 공식 표현을 추가하는 데 도움을 받았다. 또한 AES-CBC에서 구현 방식을 이미지로 보여주고 싶어서 사용한 `st.image`를 도움 받았습니다.
- AI는 암호화 로직의 핵심 아이디어를 직접 구현한 것이 아니라, 이미 구현된 기능의 사용자 인터페이스를 꾸미고 코드의 가독성을 높이는 보조적인 역할로 활용되었습니다.
- 비즈네르 암호와 AES-CBC 부분을 구성 할 때 CBC는 방식을 몰입보았습니다. 비즈네르의 경우 반복치환 과정에서 오류가 계속 떠서 물어 봤습니다.
- 조합부분에서 순서를 착각하여 오류가 나던 부분을 못찾고 있었는데 AI를 통해 실수를 잡았습니다.